Three.js + WebView2 1週間学習ガイド

Oct 16, 2026 · @shouya hashimoto

全体像と1週間の進め方

1週間のゴールは「サンプルのコードを自分の言葉で説明でき、色・形・データ項目を自分で変えられる」ことです。ゼロから自由に作れる状態は、その先の1〜2か月で身につくものなので、まず「改造できる」を目指します。

進め方の原則

- 1日の必須は60分、発展は+30〜60分です。忙しい日は必須だけで先へ進めます。
- 毎日「読む → 壊す → 直す」を繰り返します。読むだけでは身につかないので、必ず1か所は値を変えて結果を見ます。
- Day1〜4はブラウザとテキストエディタだけで足ります。サンプルには、C#なしで動く「デモモード」があります。Visual StudioとWebView2が必要なのはDay5からです。

ブラウザだけで動かす方法

ES Modules(`import`)は `file://` で開くと動かないため、簡易サーバーを使います。

```
cd WarehouseHeatmap/wwwroot
python -m http.server 8000
```

ブラウザで `http://localhost:8000/` を開き、F12で開発者ツールを出します。Pythonがなければ、VS Codeの拡張機能「Live Server」でも同じことができます。

7日間の一覧

Day	テーマ	主に読む場所	終わったときにできること
1	CSS(配置・装飾)	index.html の style	パネルを好きな位置に置き、見た目を変えられる
2	モダンJavaScript	app.js の全体構造	import・Map・イベントのコードを読める
3	Three.js基礎	app.js の「シーン」部分	箱を置いて回して見せるコードを書ける
4	Three.js応用	app.js の棚・色・クリック	色分けとクリック判定の仕組みを説明できる
5	WebView2とVB.NET	MainForm.vb、WebBridge.vb	C#とJSでメッセージをやり取りできる

Day	テーマ	主に読む場所	終わったときにできること
6	データの流れとデバッグ	全ファイル	項目を1つ追加して画面まで通せる
7	自由課題	なし	自分で決めた機能を1つ作る

Day1 CSS: 配置・並べ方・重なり

「色とbox」の次に必要なのは、どこに置くか(position)、どう並べるか(flex)、何が上に重なるかの3つです。サンプルの画面は、ほぼこの3つだけで組んでいます。
`wwwroot/index.html` の `<style>` を横に開いて読んでください。

学習のコツは、ブラウザのF12(開発者ツール)で Elements タブを開き、右の Styles 欄の値を直接書き換えて見ることです。ファイルを保存しなくても結果が即座に見えるので、試行錯誤が速くなります。

1. CSS変数(`:root` と `var()`)

```
:root {
  --panel-bg: rgba(255, 255, 255, 0.92);
  --accent: #2563eb;
}
.panel { background: var(--panel-bg); }
button.tool.active { background: var(--accent); }
```

- `--名前` で変数を定義し、`var(--名前)` で使います。`:root` に書くとページ全体で使えます。
- 色を変数にすると、テーマ色の変更が1か所で済みます。
- `rgba(赤, 緑, 青, 透明度)` の最後は 0(透明)〜1(不透明)です。パネルの 0.92 は「後ろがうっすら透ける」程度です。

2. `box-sizing: border-box`

```
* { box-sizing: border-box; }
```

- 既定(`content-box`)では、`width` は中身だけの幅です。実際の幅 = width + padding + border になり、計算がずれます。
- `border-box` にすると、`width: 270px` は padding と border を含めて 270px になります。WinFormsのコントロールの `Width` と同じ感覚です。
- サンプルでは全要素に適用しています。

3. `position: absolute` と `inset`

```
#view { position: absolute; inset: 0; }
#toolbar { top: 10px; left: 10px; }
#legend  { left: 10px; bottom: 10px; }
#banner  { top: 10px; right: 10px; }
```

- `absolute` は通常の並び(上から順に積む流れ)から外れ、基準の箱からの距離で場所を決めます。
- 基準の箱 = position 指定のある一番近い親。無ければ画面(ビューポート)です。サンプルではどの親にも position を指定していないので、画面が基準です。
- `inset: 0` は `top / right / bottom / left` を全部 0 にする省略形です。基準の箱いっぱいに広がるので、3D表示の場所に使っています。
- absolute の要素は親の高さに影響しません。親が中身で高さを決める設計には向きません。
- `html, body { height: 100%; overflow: hidden; margin: 0; }` で、画面いっぱいに使い、スクロールバーを出さないようにしています。

4. 重なり順(z-index)

- サンプルでは `z-index` を使っていません。`position` 指定のある要素同士では、HTMLで後ろに書かれた要素が上に重なるからです。
- `#view`(3D) を先に、パネル類を後に書いているので、パネルが3Dの上に載ります。
- 順番を変えたいときだけ `z-index: 10;` のように数字を付けます。大きいほうが上です。

5. `display: flex` (横一列の並べ方)

```
#toolbar { display: flex; gap: 14px; align-items: center; }
#legend .ticks { display: flex; justify-content: space-between; }
```

- `display: flex` を親に付けると、子要素が横一列に並びます(WinFormsの `FlowLayoutPanel` に近い)。
- 並べる向きを「主軸」、それと直角の向きを「交差軸」と呼びます。既定では主軸が横です(`flex-direction: row`)。

プロパティ	効果	サンプルでの使いどころ
`gap`	子同士のすき間	ツールバーのボタン間隔
`align-items`	交差軸(縦)の揃え方	ラベルとボタンの高さを中央揃え
`justify-content`	主軸(横)の分配	凡例の目盛りを左右両端に分散

6. `pointer-events: none`

```
#tip { pointer-events: none; }
.zone-label { pointer-events: none; }
```

- 要素を、マウス操作に対して「透明」にします。クリックやホバーが、下にある3D画面に届きます。
- ツールチップはマウスのすぐ横に出ます。これが無いと、ツールチップがマウスの下に入って 3D側のイベントが途切れ、表示がちらつきます。

7. `transform` と `will-change`

```
.zone-label { position: absolute; left: 0; top: 0; will-change: transform; }
```

```
el.style.transform = `translate(${x}px, ${y}px) translate(-50%, -50%)`;
```

- `translate(x, y)` で、要素を左上基準からずらします。`translate(-50%, -50%)` は「自分の大きさの半分、戻す」で、中心を座標に合わせる定番の書き方です。
- 3Dのラベルは毎フレーム位置が変わります。`left/top` を変えるより `transform` を変えるほうが、レイアウトの再計算が起きないで軽いです。
- `will-change: transform` は「これから頑繁に動かす」とブラウザに伝えるヒントです。

8. 見た目の装飾

```
box-shadow: 0 2px 8px rgba(0, 0, 0, 0.08);
border-radius: 8px;
text-shadow: 0 0 3px #fff, 0 0 3px #fff;
```

- `box-shadow: 横 縦 ぼかし 色` で影を付けます。パネルを背景から浮かせています。
- `border-radius` は角丸です。
- `text-shadow` を白で重ねると、文字のまわりに白い縁取りができます。棚の色がどんなでもゾーン名が読めます。

9. セレクタの書き方と優先度

書き方	意味
`#legend`	idが legend の要素
`.panel`	classが panel の要素
`#legend .title`	legend の中にある、classが title の要素(子孫全部)
`.group > span`	group の直接の子の span だけ
`button.tool.active`	classが tool と active の両方を持つ button
`button:hover`	マウスが乗っているときの状態

- 同じ要素に複数のルールが当たるときは、「詳細度」が高い方が勝ちます。目安は id > class > 要素名の順です。同じ詳細度なら後に書いた方が勝ちます。
- 「色を変えたのに反映されない」ときは、ほとんどこの優先度が原因です。F12の Styles 欄で、取り消し線が引かれたルールが負けているものです。

つまずきやすい点

- `absolute` の要素の基準は「position 指定のある一番近い親」です。サンプルを、余白のあるレイアウトの中に組み込むと位置がずれるのはこれが原因です。親に `position: relative` を付けると基準が親になります。
- 3Dのキャンバスの色(背景)は CSSではなく JavaScript(`scene.background`)で指定しています。CSSで背景色を変えても、キャンバスの中は変わりません。

演習(各約15分)

1. ツールバーを右上に移す(`left` を `right` に変える)。
2. 凡例の幅を 400px にし、色バーの高さを 20px にする。
3. `--accent` を緑系の色に変え、ボタンの選択色が変わるのを確かめる。
4. `#tip` から `pointer-events: none` を消して、棚の上で表示がどう変わるか観察する。見たあと元に戻す。
5. (発展) 凡例パネルを画面の右下に移し、さらに「パネルの開閉ボタン」を作る(ボタンで `display: none` を切り替える)。JavaScriptは Day2 の後でも構いません。

Day2 モダンJavaScript: app.jsを読むための前提

`app.js` は約450行ですが、使っている文法は約10種類です。Three.jsのAPIを覚える前に、この10種類を読めるようにします。VB.NET/C#の経験があるので、「あれのこれに相当する」という対応表で覚えるのが最短です。

1. ES Modules と importmap

```
<script type="importmap">
  { "imports": { "three": "./lib/three.module.js" } }
</script>
<script type="module" src="./app.js"></script>
```

```
import * as THREE from 'three';
import { OrbitControls } from './lib/OrbitControls.js';
```

- `type="module"` のスクリプトは、ファイルごとに独立したスコープを持ちます。変数がグローバルに漏れません。
- HTMLの解析が終わってから実行され、strict mode で動きます。
- `file://` で開くとセキュリティ制限で読めません。Day1で使った簡易サーバーや、WebView2の仮想ホスト(Day5)が必要な理由です。
- importmap は「`'three'` という名前 → このファイル」の対応表です。`OrbitControls.js` も内部で `from 'three'` と書いているので、この表がないと動きません。
- `three.module.js` は同じフォルダの `three.core.js` を相対パスで import します。`lib/` のファイルはバラで動かさず、セットで置いてください。
- `import * as THREE` は全部を `THREE.名前` で使う書き方、`import { OrbitControls }` は必要な名前だけを取り出す書き方です。

2. const / let / アロー関数 / テンプレートリテラル

```
const state = { shelves: [], max: 0 };   // 再代入しない
let shelfMesh = null;                     // 再代入する
const denormalize = (t) => t * state.max; // アロー関数
el.style.transform = `translate(${x}px, ${y}px)`;
```

- `const` は「変数を別の値に差し替えられない」だけです。`state.max = 5` のように中身を変えるのはできます(VB.NETの `ReadOnly` フィールドに近いです)。
- 迷ったら `const` を使い、再代入が必要なときだけ `let` にします。`var` は使いません。
- `(x) => x * 2` はラムダ式です(VB.NETの `Function(x) x * 2`)。

- バックティックで囲んだ文字列の `${式}` は、VB.NETの補間文字列 `$"...{x}..."` と同じです。

3. オブジェクト・分割代入・スプレッド

```
const [t0, c0] = STOPS[k - 1];                      // 配列の分解
function selectKey(key, { fly = false } = {}) {} // 省略可能な名前付き引数
for (const obj of [...group.children]) {}          // 配列のコピー
```

- `[a, b] = 配列` は、配列の要素を変数に分けて入れる書き方です。
- `{ fly = false } = {}` は、省略できる名前付き引数です。呼び出しは `selectKey(key, { fly: true })` です(VB.NETのオプショナル引数 + 名前付き引数に近い)。
- `[...配列]` は配列のコピーです。ループ中に要素を削除すると元の配列が変わって順番が狂うので、コピーを回しています。

4. Map と配列メソッド(LINQ相当)

```
state.codeIndex = new Map(state.shelves.map((s, i) => [s.code, i]));
state.heat = new Map((msg.items || []).map((it) => [it.code, it]));
```

上のコードは、「棚コード → 位置」と「棚コード → 集計結果」の辞書を作っています。

JavaScript	VB.NET
`new Map()` / `.get(k)` / `.set(k, v)` / `.has(k)`	`Dictionary(Of K, V)` / `TryGetValue` / `(k) = v` / `ContainsKey`
`arr.map(f)`	`arr.Select(f)`
`arr.filter(f)`	`arr.Where(f)`
`arr.find(f)`	`arr.FirstOrDefault(f)`
`arr.forEach(f)`	`For Each`
`a === b`	`a = b` (JSでは `==` でなく必ず `===` を使います)
`null` / `undefined`	`Nothing`
`a ?? b`	`If(a, b)`
`a || b`	`If(a, b)` に近いが、`0` や空文字でも右が選ばれる
`obj?.x`	`obj?.x` (同じ)

5. イベントと単一スレッド

```
renderer.domElement.addEventListener('pointermove', (ev) => { /* ... */ });
host.addEventListener('message', (ev) => onMessage(ev.data));
```

- `addEventListener('名前', 関数)` は、VB.NETの `AddHandler` です。`ev` が `EventArgs` に当たります。
- JavaScriptは基本的にシングルスレッドです。イベントは1つずつ順に処理されるので、重い処理を書くと画面が固まります(WinFormsのUIスレッドと同じです)。
- このサンプルの `app.js` には `async/await` は出てきません。概念は VB.NET の `Async/Await` とほぼ同じで、`Promise` が `Task` に当たります。
- クリックとドラッグの区別もイベントで書いています。`pointerdown` と `pointerup` の間の移動量が5px以下ならクリック、超えたら視点操作とみなしています。

6. requestAnimationFrame と ResizeObserver

```
function loop(now) {
  requestAnimationFrame(loop);              // 次のフレームも予約
  const dt = Math.min(0.1, (now - last) / 1000);
  last = now;
  /* アニメーションを dt 秒ぶん進める */
  renderer.render(scene, camera);
}
requestAnimationFrame(loop);
new ResizeObserver(resize).observe(container);
```

- `requestAnimationFrame(f)` は「次の画面更新の直前に f を呼んで」という予約です。自分で再度呼ぶとループになり、通常は毎秒60回回ります。
- タイマー(`Timer`)と違い、画面更新に同期し、タブが非表示のときは止まります。省電力です。
- `now` はミリ秒の時刻です。`dt` (前フレームからの経過秒)に比例して動かすと、PCの速さに関係なく同じ速度になります。`Math.min(0.1, ...)` は、タブに戻った直後に dt が巨大になるのを防ぐあらかじめの制限です。
- `ResizeObserver` は、要素の大きさの変化を監視します(WinFormsの `Resize` イベントに近い)。WebView2の左右分割バーを動かしたときも追従します。

7. DOM操作(HTMLをJavaScriptから変える)

```
const tip = document.getElementById('tip');
tip.replaceChildren();                        // 中身を空にする
const d = document.createElement('div');  // 要素を作る
d.textContent = '入庫';                    // 文字を入れる
tip.appendChild(d);                           // 追加する
tip.style.display = 'block';                  // CSSを直接変える
b.classList.toggle('active', b.dataset.view === view);
```

- 文字を入れるのに `innerHTML` ではなく `textContent` を使っています。`innerHTML` は文字列をHTMLとして解釈するので、CSV由来の品目名に `<` が入っていると表示が壊れる、または悪用される原因になります。
- `data-view="iso"` のような属性は、JavaScriptから `dataset.view` で読めます。ボタンの識別に便利です。
- `classList.toggle('active', 条件)` でCSSクラスを付け外しし、見た目はCSS(Day1)に任せます。「状態はJS、見た目はCSS」と分けるのが定番です。

8. 真偽値と型

- JavaScriptで `false` として扱われる値は、`false` / `0` / `''` / `null` / `undefined` / `NaN` です。`if (!h || h.v <= 0)` のように、「無い」をまとめて判定できます。
- 数値はすべて倍精度小数(`Double`)です。`Integer` や整数除算 `\` はありません。整数にしたいときは `Math.round()` を使います。

9. 開発者ツールでコードを動かしながら読む

- Console タブ: `console.log(変数)` の出力と、エラーが見えます。ここに直接 `state` と打つと、現在の状態が見られます。
- Sources タブ: `app.js` を開き、行番号をクリックするとブレークポイントになります。Visual Studioと同じようにステップ実行と変数の確認ができます。
- JavaScriptは実行するまで型エラーが出ません。赤いエラーを読む習慣が大切です(`x is not a function` は型や名前の間違いです)。

演習(各約15分)

1. `onMessage` の先頭に `console.log(msg.type, msg)` を入れ、Consoleで `layout` と `heat` の中身を観察する。
2. Sources タブで `applyHeat` の中にブレークポイントを置き、`c`(色)の値が棚ごとに変わるのをステップ実行で確かめる。
3. `normalize` の `sqrt` を `Math.cbrt`(3乗根)に変える。凡例の目盛りがずれるので、なぜずれるか考えて `denormalize` を直す(今回の逆関数は3乗)。

4. (発展) ツールバーにボタンを `data-scale="pow2"` で追加し、`normalize` と `denormalize` に `case 'pow2'` を足す。クリック処理は既存のものがそのまま使えます。

Day3 Three.js基礎: 箱を1つ置いて見せる

Three.jsの画面は、シーン・カメラ・レンダラーの3つの部品でできています。まず約30行の最小コードで箱を1つ表示し、そのあと `app.js` の「シーン」部分と見比べます。

最小コード

`wwwroot/hello.html` として保存し、`http://localhost:8000/hello.html` で開きます (`lib/` と同じ階層に置くこと)。コピペではなく、一度は手で打ってください。

```
<!doctype html>
<html><head><meta charset="utf-8">
<style>html, body { margin: 0; overflow: hidden; }</style>
<script type="importmap">{ "imports": { "three": "./lib/three.module.js" } }
</script>
</head><body>
<script type="module">
import * as THREE from 'three';
import { OrbitControls } from './lib/OrbitControls.js';

const renderer = new THREE.WebGLRenderer({ antialias: true });
renderer.setSize(innerWidth, innerHeight);
document.body.appendChild(renderer.domElement);

const scene = new THREE.Scene();
const camera = new THREE.PerspectiveCamera(45, innerWidth / innerHeight, 0.1,
1000);
camera.position.set(6, 5, 8);

const box = new THREE.Mesh(
  new THREE.BoxGeometry(2, 1, 1),
  new THREE.MeshStandardMaterial({ color: 0x2563eb })
);
scene.add(box);
scene.add(new THREE.HemisphereLight(0xffffff, 0x888888, 2));
scene.add(new THREE.GridHelper(20, 20));

const controls = new OrbitControls(camera, renderer.domElement);
(function loop() {
  requestAnimationFrame(loop);
  controls.update();
```

```
  renderer.render(scene, camera);
})();
</script>
</body></html>
```

3つの部品

部品	役割	WinFormsでのたとえ
`Scene`	描く物を入れる空間(木構造)	`Controls` のツリー
`Camera`	どこから、どの範囲を見るか	表示領域(ビューポート)
`WebGLRenderer`	scene と camera から canvas に絵を描く	`Paint` での描画処理

- `renderer.render(scene, camera)` は、呼んだ瞬間の1枚だけを描きます。自動で再描画はしないので、ループで毎フレーム呼びます。
- サンプルも同じ構造です。`app.js` の「シーン」という見出しのあたりを開くと、同じ `renderer` / `scene` / `camera` が作られています。

Mesh = Geometry + Material

- `Geometry` は形です。`BoxGeometry(幅, 高さ, 奥行)`、`PlaneGeometry`、`CylinderGeometry`、`SphereGeometry` などがあります。
- `Material` は見た目です。よく使うのは次の3つです。

Material	光の計算	サンプルでの使い方
`MeshBasicMaterial`	なし(色がそのまま出る)	床、選択ハイライト
`MeshLambertMaterial`	簡易(軽い)	棚(数百〜数千個を軽く描く)
`MeshStandardMaterial`	物理ベース(リアル、やや重い)	設備モデルを作る段階で検討

- `Mesh` は `Object3D` を継承しています。`position`、`rotation`、`scale`、`visible`、`add()` / `remove()` はすべての3Dオブジェクトで同じように使えます。
- `Group` は中身のない入れ物です。サンプルの `worldGroup` は、「床・棚・枠」をまとめて消すために使っています。
- Geometry / Material / Texture は GPU メモリを使います。捨てるときは `.dispose()` が必要です(Day4)。

座標系: ここで一番迷います

- Three.jsは右手系で、**Yが上**、Xが右、Zが手前(画面からこちら側)です。
- 図面やCADは Z が高さ、Y が北です。サンプルの棚マスタもこちらです。
- そのため `app.js` では、表示するときに `(x, y, z) = (X, Z, -Y)` と変換しています。北が奥(`-Z`)になるように符号を反転しています。

```
// app.js: 棚1個の位置(箱は中心基準なので、下端の Z に高さの半分を足す)
p.set(s.x, s.z + s.h / 2, -s.y);
```

- `Box` の `position` は箱の**中心**です。マスタの Z は「下端」なので、`+ h / 2` が要ります。これを忘れると棚が床にめり込みます。
- 回転はラジアンです。`THREE.MathUtils.degToRad(90)` で度から変換できます。
- `PlaneGeometry` は初期状態で立っています。床にするには `floor.rotation.x = -Math.PI / 2` で寝かせます(サンプルの `rebuildWorld` にあります)。
- 単位は自由ですが、サンプルは 1 = 1m としています。

カメラ

```
const camera = new THREE.PerspectiveCamera(45, aspect, 0.1, 2000);
//                                          縦の視野角, 縦横比, near, far
```

- near より近い物と、far より遠い物は描かれません。「急に消える」ときはここを疑います。
- ウィンドウのサイズが変わったときは、`camera.aspect = w / h;` と `camera.updateProjectionMatrix();` をセットで呼びます(サンプルの `resize()`)。
- 遠近感のない真上図が欲しい場合は `OrthographicCamera` を使います。サンプルは `PerspectiveCamera` で真上から見ています。
- `setView` の距離の計算は三角関数です。「見たい範囲の半分 ÷ tan(視野角の半分)」で、ちょうど画面に収まる距離になります。

光

- `MeshBasicMaterial` 以外は、光が無いと真っ黒です。「色を指定したのに黒い」はほぼこれが原因です。
- `HemisphereLight(空の色, 地面の色, 強さ)` は、上からやわらかく全体を照らす光です。`DirectionalLight` は太陽光です。位置は「光が来る方向」だけに意味があります。
- r155以降は光の強さが物理単位になり、古い記事より大きめの数字が必要です。ネットの古いサンプルで「暗い」ときは強さを上げてみてください。

OrbitControls

- 左ドラッグ = 回転、右ドラッグ = 平行移動、ホイール = ズームです。
- `controls.target` が「回転の中心」です。カメラの向きは `lookAt` ではなく、ここで決まります。
- `enableDamping = true` で慣性が付きます。そのときは毎フレーム `controls.update()` を呼ぶ必要があります。
- `maxPolarAngle = Math.PI / 2 - 0.02` で、床の下に潜らないようにしています。
- カメラ位置をコードで動かしたときは、`controls.update()` で整合を取ります(`setView`、`flyTo`)。

レンダラー設定

- `renderer.setPixelRatio(Math.min(devicePixelRatio, 2))` は、高解像度ディスプレイでのくっきり感と負荷のバランスです。上限を 2 にしています。
- `renderer.setSize(w, h)` は、既定で canvas のCSSサイズも変えます。
- サンプルは常にループを回しています。変化があったときだけ `render()` する方式にすると、ノートPCで省電力になります(発展)。

「何も映らない」ときのチェックリスト

1. `scene.add(...)` を忘れていないか。
2. `renderer.render(scene, camera)` を呼んでいるか(ループの中にあるか)。
3. カメラが箱の中や、箱の真裏にいないか。near / far の範囲外ではないか。
4. Lambert / Standard を使っていて、光を入れていないか。
5. canvas の親要素の高さが0になっていないか(Day1の `height: 100%` や `inset: 0`)。
6. Consoleタブに赤いエラーが出ていないか(`import` のパス間違いが多いです)。

演習(各約15分)

1. `hello.html` を自分で打ち、箱の色とサイズを変える。
2. 箱を3つ並べる(`position.x` を変える)。1つを `rotation.y = Math.PI / 4` で斜めにする。
3. 床を足す(`PlaneGeometry(20, 20)` + `rotation.x = -Math.PI / 2`)。
4. `scene.background = new THREE.Color(0xeef1f5)` を足し、光を消して見え方を確かめる。その後 `MeshBasicMaterial` に変えて、違いを見る。
5. 図面座標の棚(X=3, Y=5, 下端Z=1.5, 高さ=1.2)を置く。答えは `position.set(3, 1.5 + 0.6, -5)` です。この式を自分で導けるか確認してください。

6. (発展) `for` ループで10×3の棚を個別の `Mesh` として並べる。これがDay4の `InstancedMesh` でどう楽になるかの比較基準になります。

Day4 Three.js応用: app.jsの棚・色・クリック

今日は、サンプルの中核である「棚を大量に描く」「値を色にする」「クリックで選ぶ」を、`app.js` を読める状態にします。この3つがあれば、在庫・設備・センサーなどどんなデータも同じ型で表示できます。

1. InstancedMesh: 同じ形をまとめて描く

```
shelfMesh = new THREE.InstancedMesh(
  new THREE.BoxGeometry(1, 1, 1),
  new THREE.MeshLambertMaterial({ color: 0xffffff }),
  state.shelves.length            // 個数
);
state.shelves.forEach((s, i) => {
  p.set(s.x, s.z + s.h / 2, -s.y);                  // 位置
  sc.set(s.w * 0.94, s.h * 0.92, s.d * 0.94);       // 大きさ(隣と少し隙間を空ける)
  m.compose(p, q, sc);                              // 位置+回転+大きさ → 4x4行列
  shelfMesh.setMatrixAt(i, m);
  shelfMesh.setColorAt(i, COLOR_ZERO);
});
shelfMesh.instanceMatrix.needsUpdate = true;
```

- 個別の `Mesh` を360個作ると、GPUへの描画命令(draw call)が360回になります。`InstancedMesh` は、同じ形・同じ素材の物を**1回の命令**でまとめて描きます。数千、数万個でも軽い理由です。
- 1個ずつ違うのは「行列」と「色」だけです。行列は位置・回転・拡大縮小を1つの4x4の数字の表にまとめたもので、中身を理解しなくても `compose(位置, 回転, 大きさ)` で作れます。
- 1×1×1の箱を `scale` で伸ばして、棚ごとの大きさにしています。回転はありません。回転はクォータニオン(`Quaternion`)という4つの数で表し、`new THREE.Quaternion()` は「回転なし」です。
- `p`、`q`、`sc`、`m` をループの外で作って使い回すのは、ループ内で `new` すると大量のゴミ(GC)が出てカクつくからです。
- Materialの色を白(`0xffffff`)にしているのは、個別の色がそのまま乗算されて見えるようにするためです。

- 一つの `InstancedMesh` で扱えるのは同じGeometryとMaterialだけです。棚とパレットなど形が違う物は、別の `InstancedMesh` にします。
- 行列を設定したあとは、当たり判定の範囲(境界球)を `computeBoundingSphere()` で計算し直します。位置を後から動かしたときも同じです。

2. 色の変更と「再転送フラグ」

```
shelfMesh.setColorAt(i, c);
// ...全部の色を設定した後で
shelfMesh.instanceColor.needsUpdate = true;
```

- 位置や色は、まず CPU 側の配列に書かれ、GPUへの転送は「`needsUpdate = true`」を立てたときだけ行われます。
- 値を変えたのに画面が変わらないときの第一候補はこのフラグの立て忘れです。Three.jsでよくあるトラブルです。

3. ヒートマップの色を作る

```
const STOPS = [
  [0.00, '#2c7bb6'], [0.25, '#abd9e9'], [0.50, '#ffffbf'],
  [0.75, '#fdae61'], [1.00, '#d7191c'],
].map(([t, c]) => [t, new THREE.Color(c)]);

function rampColor(t, target) {
  for (let k = 1; k < STOPS.length; k++) {
    if (t <= STOPS[k][0]) {
      const [t0, c0] = STOPS[k - 1];
      const [t1, c1] = STOPS[k];
      return target.copy(c0).lerp(c1, (t - t0) / (t1 - t0));
    }
  }
}
```

- ヒートマップは「0〜1の値 → 色」の関数です。色の停止点(STOPS)を並べ、値が入る区間を探して、区間内の割合で補間します。
- `new THREE.Color('#2563eb')`、`new THREE.Color(0x2563eb)`、`new THREE.Color('red')` を使えます。`a.lerp(b, r)` は a と b の間を割合 r で補間します。
- 青→水色→蘇黄→橙→赤は、ColorBrewerの RdYlBu を反転した配色で、赤と緑だけで分けないため色覚の違いでも読みやすくなります。
- 0回は別の灰色にしています。「データなし」と「少ない」を混同しないためです。

値を 0〜1 にする(正規化)と尺度

尺度	式	向いているとき
線形	`v / max`	値がほぼ均等に分布している
平方根	`sqrt(v / max)`	一部の棚に集中している(今回の既定)
対数	`log1p(v) / log1p(max)`	桁違いに分布している

- 入出庫回数は「少数の棚に集中する」分布になりやすいです。線形だと大半が青一色になるため、平方根や対数で中間の違いを見やすくします。
- 凡例の目盛りは、逆変換(`denormalize`)で「色の位置→回数」を計算しています。尺度を足すときは、必ず逆変換もセットで足します。
- 色は見た人の判断のためにあります。凡例を必ず付け、「何の量を色にしているか」を画面に出してください。

4. Raycaster: クリックした棚を判定する

```
pointer.set(((ev.clientX - r.left) / r.width) * 2 - 1,
           -((ev.clientY - r.top) / r.height) * 2 + 1);
raycaster.setFromCamera(pointer, camera);
const hit = raycaster.intersectObject(shelfMesh)[0];
if (hit && hit.instanceId !== undefined) { /* hit.instanceId 番目の棚 */ }
```

- 画面のクリック位置から、カメラの奥へ光線(ray)を伸ばし、当たった物を近い順の配列で返します。`[0]` が一番手前です。
- `pointer` は「正規化デバイス座標」です。画面中央が (0, 0)、左上が (-1, 1)、右下が (1, -1) です。**Yの向きがマウス座標と逆**なので、符号の反転があります。
- `InstancedMesh` の場合、`hit.instanceId` が「何番目の棚か」を示します。`state.shelves[instanceId]` → 棚コード → `state.heat` (Map) → 集計キーと辿ります。
- 数万個以上では、`pointermove` で毎回判定すると重くなりうるので、間引き(スロットル)を検討します。

5. Box3 と選択ハイライト

```
const box = new THREE.Box3();
for (const i of idxs) { /* 棚ごとの箱を tmp に作り */ box.union(tmp); }
selFill.position.copy(box.getCenter(new THREE.Vector3()));
selFill.scale.copy(box.getSize(new THREE.Vector3()));
selLines.box.copy(box);
```

- `Box3` は軸に平行な直方体(最小点と最大点)です。`union` で複数を包む箱に拡張します。「間口単位」で選ぶときに、同じ間口の棚(3段)をまとめて囲んでいます。

- `Box3Helper` は箱を線で描きます。半透明の箱は `transparent: true, opacity: 0.16, depthWrite: false` です。
- `depthWrite: false` は、透明な物が後ろの物を隠さないための定番の設定です。

6. 3D座標→画面座標(`project`)とHTMLラベル

```
v3.copy(z.pos).project(camera);          // -1〜1 の正規化座標になる
if (v3.z > 1) { z.el.style.display = 'none'; continue; } // カメラの後ろ
const sx = (v3.x * 0.5 + 0.5) * w;
const sy = (-v3.y * 0.5 + 0.5) * h;
z.el.style.transform = `translate(${sx}px, ${sy}px) translate(-50%, -50%)`;
```

- 「3Dの点」を「画面のどこに見えるか」に変換して、HTML要素をそこに置いています。Day1の `transform` とつながります。
- 3D文字(`TextGeometry` や `Sprite`)より、HTMLのほうが鮮明で、CSSで装飾もできます。数百個以上のラベルは DOM が重くなるので、表示するものを絞るか、別の方法を検討します。

7. カメラを滑らかに動かす(`flyTo`)

```
flight.t = Math.min(1, flight.t + dt * 2.5);
const k = 1 - Math.pow(1 - flight.t, 3);          // easeOutCubic
controls.target.lerpVectors(flight.from, flight.to, k);
camera.position.copy(controls.target).add(flight.offset);
```

- `t` を 0→1 に毎フレーム進め、`k` を使って始点と終点を補間します。`dt` を掛けることで、PCの速さに関係なく約0.4秒(1 ÷ 2.5)で終わります。
- `easeOutCubic` は、最初は速く、最後はゆっくり止まる動きです。式を変えるだけで動きの印象が変わります。
- `offset` を保っているのは、「カメラと注視点の相対位置」を変えずに平行移動するためです。見る角度や距離はそのままです。

8. メモリ解放(`dispose`)

```
function disposeGroup(group) {
  for (const obj of [...group.children]) {
    group.remove(obj);
    obj.traverse((o) => {
      if (o.geometry) o.geometry.dispose();
      if (o.material) o.material.dispose();
    });
  }
}
```

- JavaScriptのゴミ回収(GC)は、GPUメモリを解放しません。Geometry、Material、Texture は、捨てるときに `.dispose()` を呼ぶ必要があります。
- サンプルは「棚の配置が変わるたびに作り直す」設計なので、これが無いと CSV再読込を繰り返すたびにGPUメモリが増え続けます。
- 他で共有中の Geometry / Material を dispose すると、そちらが壊れます。共有している物だけは捨てないでください。

9. 設計の考え方: 状態(state)と描画の分離

- `app.js` は、受け取ったデータを `state` に入れ、`applyHeat()` が `state` から見た目を作り直します。メッセージ処理は「stateを更新して描画関数を呼ぶ」だけです。
- 「データが変わったら、見た目を全部やり直す」という単純な方針です。差分を考えるより間違いが起きにくく、360棚では十分速いです。
- この分離のおかげで、集計(C#)と表示(JS)を別々に直せます。

演習(各約15〜20分)

1. `STOPS` を 3色(青・黄・赤)に減らして、見やすさの変化を比べる。
2. `COLOR_ZERO` と選択枠の色を変える。
3. `app.js` の末尾近くに `window.__renderer = renderer;` を一時的に足し、Consoleで `__renderer.info.render.calls`(描画命令の回数)と `__renderer.info.memory`(メモリの個数)を見る。360棚でも描画命令が数回で済むことを確かめます。
4. Day3発展の「個別Meshで並べた版」でも同じ値を見て、描画命令の数の差を確認する。
5. (発展) 50回を超えた棚だけ赤、他は灰色にする「しきい値モード」を `applyHeat` に足す。
6. (発展) マウスを乗せた棚を少し浮かせる(`setMatrixAt` の y を変えて `instanceMatrix.needsUpdate = true`)。

Day5 WebView2 と VB.NET: C#とJSをつなぐ

今日から Visual Studio を使います。今日の要点は「WebView2 は別プロセスのブラウザで、C# とは JSON のメッセージでしか話せない」ということです。この前提を理解すると、コードのあらゆる場所の理由が説明できます。

1. WebView2 とは何か

- Microsoft Edge(Chromium)をアプリに埋め込むコントロールです。WinFormsでは NuGet の `Microsoft.Web.WebView2` を入れると、`WebView2` コントロールが使えます。
- **Evergreen Runtime**: PCに入っている WebView2 Runtime を共有して使います。自動で更新され、Windows 11 には標準で入っています。Windows 10 は入っていないPCがあるので、社内配布前に確認してください。
- **Fixed Version**: 特定バージョンのランタイムをアプリと一緒に配布する方式です。バージョンを固定したい工場のオフラインPC向けですが、更新は自分で管理します。サンプルは Evergreen を使っています。
- **プロセス構造**: アプリ(`WarehouseHeatmap.exe`)とは別に、`msedgewebview2.exe` が複数起動します(ブラウザ・レンダラー・GPUなど)。JavaScript はレンダラープロセスで動き、.NETのオブジェクトとはメモリを共有しません。タスクマネージャーでも確かめられます。
- **ユーザーデータフォルダ**: キャッシュや Cookie や localStorage の保存先です。既定では exe の隣に作るため、`Program Files` 配下では書き込み権限がなくて失敗します。サンプルは `%LOCALAPPDATA%\WarehouseHeatmap\WebView2` を明示しています。

2. 初期化は非同期(MainForm.vb の InitWebAsync)

```
Dim env As CoreWebView2Environment = Await
CoreWebView2Environment.CreateAsync(Nothing, userData)
Await _web.EnsureCoreWebView2Async(env)
```

- ランタイムのプロセス起動に時間がかかるため、`_web.CoreWebView2` はコントロールを作った直後は `Nothing` です。`EnsureCoreWebView2Async` が終わってから使います。
- `Await` の前後でも、スレッドは UI スレッドに戻ります(WinForms の同期コンテキスト)。このままコントロールを触って構いません。`Task.Run` の中から WebView2 を触るとエラーになります。
- `Protected Overrides Async Sub OnLoad` のような `Async Sub` は、例外が呼び出し元に届きません。サンプルで、中に `Try/Catch` を書いているのはそのためです。
- Runtime が無いPCではここで例外になるので、`Catch` で「Runtime をインストールしてください」を出しています。

3. 仮想ホスト名マッピング

```
core.SetVirtualHostNameToFolderMapping("app.local", wwwroot,
CoreWebView2HostResourceAccessKind.Allow)
core.Navigate("https://app.local/index.html")
```

- 「wwwroot フォルダを、`https://app.local/` として見せる」設定です。WebView2 の内部で名前を対応付けるだけなので、DNS やネットワークは使いません。
- 使う理由は、`file://` では ES Modules(`import`)が読めないからです(Day2)。
- `HostResourceAccessKind` は、`Allow`(どこからでも許可)、`DenyCors`(外部サイトからのクロスオリジン要求を拒否)、`Deny` です。自分のページだけを表示するこのサンプルでは `Allow` ですが、外部サイトも開くアプリでは `DenyCors` にするのが安全です。
- 代替手段と比べると、仮想ホストが最も簡単です。

方法	評価
`NavigateToString`(HTML文字列を渡す)	上限が小さく(約2MB)、複数ファイルの構成に向かない
`file://`	ES Modules が読めない
ローカルに `HttpListener` サーバー	ポート管理と管理者権限が面倒
`WebResourceRequested` で自前応答	自由だが、コードが増える
仮想ホスト名マッピング	1行で済む(サンプルの方式)

4. メッセージング: 一番大事な仕組み

向き	送る側	受ける側
C# → JS	`core.PostWebMessageAsJson(json)`	`chrome.webview.addEventListener('message', ev => ev.data)`
JS → C#	`chrome.webview.postMessage(obj)`	`AddHandler core.WebMessageReceived` で `e.WebMessageAsJson`

- どちらも「JSON のデータ」だけを送ります。関数やオブジェクトの参照は渡せません(別プロセスだからです)。
- `PostWebMessageAsJson` で送った内容は、JS 側で「パース済みのオブジェクト」として `ev.data` に入ります。
- JSからオブジェクトを `postMessage` したときは、C# 側で `e.WebMessageAsJson` を使います。`e.TryGetWebMessageAsString()` は、文字列で送ったときだけ使えるメソッドです。オブジェクトに使うと例外になります。ここは間違えやすい点です。

- メッセージは「`type` + データ」で統一しています。メッセージはプロセス間の API なので、仕様を `WebBridge.vb` の先頭コメントに書いています。メッセージを足すときは、まずここを更新してください。
- 大きなデータ(数MB以上)は、分割や差分送信を考えます。大量データ向けに `SharedBuffer` という仕組みもあります。360棚なら数十KBで、問題になりません。
- C# のオブジェクトを JS から直接呼ぶ `AddHostObjectToScript` もありますが、型の制約があるため、サンプルはメッセージ方式にしています。

5. `ready` ハンドシェイク(取りこぼし防止)

```
JS:  ページ読込完了 → { type: 'ready' } を送る
C#:  ready を受け取ったら _webReady = True し、layout → heat の順に送る
```

- C# が先に送ってしまうと、JS 側のイベントリスナーがまだ無くて取りこぼします。そのため JS が「準備できた」と言ってから送ります。このような「合図」の設計はプロセス間通信の定番です。
- ページをリロード(F5)しても `ready` が再送され、データが自動で復元されます。
- `layout`(棚の配置)を先に、`heat`(回数)を後に送る順序も重要です。JS 側は「棚がある前の色」には意味がないからです(逆順でも壊れないよう `applyHeat` を layout 後にも呼んでいます)。

6. JSON の作り方(WebBridge.vb)

- `System.Web.Script.Serialization.JavaScriptSerializer` を使っています。.NET Framework に標準で、追加の NuGet が要りません。
- `Dictionary(Of String, Object)` と `List(Of Object)` を組み合わせると、そのまま JSON になります。
- `MaxJsonLength = Integer.MaxValue` は、既定の上限(約2MB)を外しています。棚が増えたときの保険です。
- 受信側の数値は `Integer` か `Decimal` か `Double` のどれになるか定まらないため、`GetString` で `Convert.ToString` しています。
- 将来、`Newtonsoft.Json` や `System.Text.Json`(NuGet) に変えると、型付きのクラスで扱えて楽になります。

7. MainForm.vb の読み方

1. `BuildUi`: Designer を使わず、コードでコントロールを作っています(サンプルをファイル1つで読めるためです)。左右分割の左が WebView2、右が表です。
2. `OnLoad`: CSV を読み、初回の集計をしてから `InitWebAsync` を呼びます。

3. `Recalculate`:「集計→Top10表→履歴表→3Dへ送信(`PushHeat`)」の順です。絞り込み条件を変えるときは全てここを通ります。
4. `_suppress` フラグ: 初期化中に `ValueChanged` が連発しても再集計しないための、WinForms でよく使うパターンです。
5. `OnWebMessage`: JS からの `ready` と `select` を受けています。
6. 選択状態:「選択された棚」をC#とJSの両方が持つと、食い違いが起きます。サンプルは「選択の決定は JS、表示は C#」と分け、Top10をクリックしたときは `focus` メッセージで JS に依頼しています。集計単位を変えるとキーの意味が変わるので、選択を解除します(`OnUnitChanged`)。

8. デバッグと配布

- `AreDevToolsEnabled = True` なので、アプリ上で F12 を押すと開発者ツールが開きます。JavaScript の例外はアプリには出ないので、「画面が白い」「何も起きない」ときはまず F12 の Console を見ます。
- Visual Studio から JS をデバッグするより、開発者ツールの Sources タブのほうが簡単です。
- 配布用には `AreDevToolsEnabled = False` にしてください。一般ユーザーにソースを見せたくない場合の考え方です。
- 配布フォルダには、exe、`WebView2Loader.dll` と `Microsoft.Web.WebView2.*.dll`(NuGet が自動でコピーします。bin の中身は丸ごと配布してください)、`wwwroot/`、`Data/` が必要です。`vbproj` の `CopyToOutputDirectory` 設定が、ビルド時に wwwroot と Data をコピーしています。

つまずきやすい点

1. **白い画面**: F12 で Console と Network を見ます。`lib/three.module.js` が 404 なら、bin に wwwroot がコピーされていません。
2. `_web.CoreWebView2` が `Nothing` で例外: `EnsureCoreWebView2Async` の完了前に触っています。
3. メッセージが届かない: `ready` 前に送った、JSON が壊れている、`type` の綴りが違う。
4. 初回起動でエラー: Runtime 未インストール、またはユーザーデータフォルダに書き込めない。
5. JS を直しても反映されない: `bin` 側のコピーを見ています。開発中は `wwwroot` の元ファイルを直し、ビルドし直すか、「ブラウザ単体で確認」するのが速いです。

演習(各約20分)

1. `OnWebMessage` の先頭で `Debug.WriteLine(e.WebMessageAsJson)` を出し、`ready` や `select` がどう届くかを出力ウィンドウで見る。

2. F12 の Console から `window.chrome.webview.postMessage({ type: 'ping', n: 1 })` を送り、`OnWebMessage` に `Case "ping"` を足して受信する。
3. フォームにボタンを1つ足し、`PostJson(WebBridge.BuildFocusJson("A-1-03"))` を呼んで、こちらから 3D を動かす。
4. 新しいメッセージ `{ type: 'notice', text: '...' }` を作り、JS 側で画面の隅にトースト表示する(HTML/CSS/JS を復習できます)。
5. (発展) ユーザーデータフォルダを削除して起動し、初回起動との違いを見る。
6. (発展) `AreDevToolsEnabled` を `False` にして、F12 が効かないことを確かめる。

Day6 データの流れとデバッグ: 項目を1つ足して画面まで通す

Day1〜5で見た部品を、今日は1本の線でつなぎます。メインの課題は「データ項目を1つ追加して、C#の集計から、JSON、JSの表示まで通す」です。この型を一度自分で通せると、今後の拡張のほとんどが同じ手順になります。

1. 起動から表示までの流れ

順	場所	何が起きるか
1	`Program.Main`	`Application.Run(New MainForm())`
2	`MainForm.OnLoad`	CSVが無ければ仮データを生成し、棚と履歴を読む（`DataRepository`）
3	`Recalculate`	期間・種別で絞り、棚ごとの回数を集計（`HeatmapService.Calculate`）。Top10表を更新
4	`InitWebAsync`	WebView2 を初期化し、仮想ホスト経由で `index.html` を開く
5	`app.js` 起動	シーンを作り、`{ type: 'ready' }` を送る
6	`OnWebMessage`	`ready` を受けて、`layout`（棚の配置）→ `heat`（回数）の順に送る
7	`onMessage`（JS）	`rebuildWorld` で棚を作り、`applyHeat` で色を付ける
8	期間を変える	`OnFilterChanged` → `Recalculate` → `heat` だけを再送
9	3Dの棚をクリック	`pointerup` → `select` を送る → `ShowHistory` が履歴を表示
10	Top10をクリック	`OnTopCellClick` → `focus` を送る → JSの `flyTo`

各行にブレークポイント(VBは該当メソッド、JSは開発者ツールの Sources)を置いて、順番どおりに止まることを確かめるのが、今日の最初の作業です。

2. 集計ロジック（`HeatmapService.Calculate`）を読む

- **回数 = 件数**です。数量（`Quantity`）の合計ではありません。数量にしたいときは、`+ 1` を `+ t.Quantity` に変えます(演習でやります)。
- 期間は `fromInclusive`（開始日の0時)以上、`toExclusive`（終了日の翌日0時)未満です。DateTimePicker の「終了日」を含めて集計するためで、終了日の23:59を取りこぼさない書き方です。
- 間口合算は、`KeyOf` が「棚コード」か「間口キー（`A-1-03`）」を返すだけで実現しています。同じキーの棚は同じ値を持ち、段が違っても同じ色になります。
- マスタに無い棚コードの履歴は無視します（`TryGetValue` 失敗で `Continue For`）。実データでは「マスタ未登録」や表記のゆれが起きやすいので、無視した件数を数えてステータスバーに出すと、親切になります。

- 1.2万件を毎回全部走査しても軽いです。数百万件になったら、DB側で `GROUP BY` して集計を任せる設計に切り替えます。

3. CSVの扱い(DataRepository.vb)

- 文字コードは UTF-8(BOM付き)です。Excel で開いても日本語が化けません。
- 日時と小数点は `CultureInfo.InvariantCulture` を明示しています。PCの地域設定(小数点がカンマの国など)によって、読み書きが壊れるのを防ぐためです。
- `ParseCsvLine` は簡易版で、セル内の改行には対応しません。実データを扱うときは、VB.NET 標準の `Microsoft.VisualBasic.FileIO.TextFieldParser` に置き換えると安全です。

4. デバッグは「層を切り分ける」

「色がおかしい」ときは、次の4層を上から順に確認します。

層	確認方法
1. データ(CSV)	Excel で開き、棚コードや日時を目視する
2. 集計(C#)	`Recalculate` の後にブレークして、`_result.MaxValue` や `TopEntries` を見る
3. メッセージ(JSON)	`PostJson` の直前で `Debug.WriteLine(json.Substring(0, 300))`
4. 描画(JS)	F12 の Console に受信内容を `console.log`。ブラウザ単体のデモでも切り分けできる

- 3が正しく、4がおかしいなら JS 側、というように、「どこまでが正しいか」を境界で判定します。メッセージ(JSON)が境界になるのが、この構成の利点です。
- JS 単体のデモモード(Day1〜4で使った方法)で「描画だけ」を先に直してから、C# とつなぐと、原因の特定が速くなります。
- Console のエラーは、右端の `app.js:123` をクリックすると該当行に飛びます。

よくある症状と原因

症状	よくある原因
3D画面が白い	`import` のパス間違い、`wwwroot` が出力フォルダにコピーされていない
棚は出るがすべて灰色	`heat` が届いていない、棚コードがマスタと履歴で一致していない

症状	よくある原因
色を変えても反映されない	`instanceColor.needsUpdate = true` の立て忘れ(Day4)
クリックが当たらない	座標変換の符号、canvas のサイズが不正、行列を変えた後の境界球の計算漏れ
日付を変えても件数が合わない	終了日の扱い(`toExclusive`)を見直す
日本語が化ける	BOM無しUTF-8を Excel で開いた、読込時の `Encoding` 指定漏れ
重い	影を使っている、`devicePixelRatio` が大きい、HTMLラベルが多い

5. メイン課題:「数量」を集計してツールチップに出す

目標は、棚にマウスを乗せたときに「入庫数量」と「出庫数量」も表示することです。この6ステップが、データ項目を足すときの型です。

1. `HeatmapService.vb` の `HeatEntry` に `InQty` と `OutQty` を足す。
2. `Calculate` の中で、件数を数えるところと同じ場所に、`t.Quantity` を足し込む Dictionary を2つ足す。
3. `WebBridge.vb` の `BuildHeatJson` に `d("iq") = e.InQty` と `d("oq") = e.OutQty` を足す。
4. `WebBridge.vb` 先頭のメッセージ仕様コメントに、`iq` と `oq` を追記する(仕様を先に直す習慣をつけます)。
5. `app.js` の `showTip` に `tip.appendChild(row('入庫数量', ...))` の行を足す。
6. 実行して、ツールチップの数字が CSV を Excel でピボット集計した値と一致するかを確かめる。

一致するかどうかの確認が、テストの基本です。最初に Excel で「1棚だけ」手計算しておくと、コードを直したときに壊れていないかがすぐわかります。

6. 発展課題

1. 上の「件数」と「数量」を切り替えるドロップダウンをフォームに足し、ヒートマップの色の元にする(`HeatEntry.Value` を切り替える形にします)。
2. 「マスタに無い棚コード」の件数をステータスバーに表示する。
3. `ParseCsvLine` を `TextFieldParser` で置き換える。
4. Git を初期化し、ここまでの変更をコミットする。変更ごとにコミットする習慣は、Claude Code で開発するときの「安全網」にもなります。

Day7 自由課題と、次の1か月

最終日は「自分で選んだ機能を1つ、最後まで作る」日です。Claude Code を使って構いません。ただし、「何を頑むか」と「出来上がりを確かめる」のは自分で行います。ここが「自由に実装できる」の実際の姿です。AI に書かせても、読める・直せるなら、自分で作ったと同じだからです。

課題候補(1つ選ぶ)

難易度	課題	触る場所	学べること
★	凡例・床・棚の隙間の色を自社カラーに合わせる	`index.html`、`app.js` の定数	CSS、色
★	期間プリセットボタン(直近7日・30日・全期間)	`MainForm.vb`	WinForms、イベント
★★	曜日別・時間帯別のヒートマップの切り替え	`HeatmapService.vb`、`MainForm.vb`	集計の拡張
★★	棚をクリックしたときに品目別ランキングを表示	`MainForm.vb`、`HeatmapService.vb`	LINQ、表示
★★	壁・柱・通路・設備を箱で追加	`app.js`	Geometry、配置
★★	滞留日数や在庫数など別の指標で色分け	`Models.vb`、`HeatmapService.vb`、`WebBridge.vb`	指標の追加(Day6の型)
★★★	3D上で棚を動かしてCSVに保存する編集モード	`app.js`、`WebBridge.vb`、`DataRepository.vb`	双方向通信、座標変換
★★★	日付スライダーで時間を進める再生	`MainForm.vb`、`app.js`	ループ、状態管理
★★★	SQL Server から直接読む	`DataRepository.vb`	ADO.NET、SQL、クエリ設計

現実的な目安は★★を目標にし、できれば余力で★★★を試す程度です。メイン課題をDay6でこなせていれば、★★の「別の指標で色分け」は同じ型の繰り返しです。

進め方(半日〜1日)

1. やることを1文で書きます。完了条件も一緒に書きます。例として、「期間プリセットボタン3つを足す。押すと日付ピッカーと集計が更新される」です。
2. どのファイルを触るかを先に予想します。Day6の流れの表を見て、どの行に関係するかを考えます。
3. Git で現在の状態をコミットしてから始めます。壊しても戻れる状態にします。
4. 1ステップ作るたびに動作確認します。まとめて作ってから動かすと、どこで壊れたか分かりません。
5. 詰まったら、まず15分は自分で調べます(Day6の「層を切り分ける」)。それでも解決しないときに、Claude Code や人に聞きます。
6. 最後に `README.md` と `CLAUDE.md` を更新します。未来の自分と、Claude Code のためのメモです。

Claude Code への頼み方

- 頼む前に、`CLAUDE.md`(サンプルに入れてあります)に構成・座標系・メッセージ仕様の場所を書いておきます。毎回説明しなくて済むし、精度も上がります。
- 良い依頼は、「目的 + 触ってよい範囲 + 完了条件 + 守ること」です。
  例:「`MainForm.vb` の上部バーに、期間プリセット(直近7日・30日・全期間)のボタンを足して。押すと `_dtFrom` と `_dtTo` を更新し、`Recalculate` が走ること。`wwwroot` は変更しない。`Option Strict On` を維持すること。」
- 悪い依頼は、「便利にして」「3Dをかっこよくして」のような、完了条件と範囲がないものです。
- 1回の依頼は1機能にします。動いたらコミットして、次の依頼に進みます。
- 出来上がりの確かめ方は3つです。① diff でどのファイル・どの行が変わったかを見る。② 実際に動かして確認する。③ 自分が説明できない行は、「この行は何をしているか」を Claude に説明させる。説明できないコードは取り込まないでください。
- Day1〜6の知識があると、「起きている問題を層で切り分けて伝える」ことができます。例えば「JSONは正しく届いているが、色が反映されない」のような依頼は、修正の精度を大きく上げます。
- 社内の実データを Claude Code に見せる前に、社内の AI 利用ポリシーを確認してください。確認できるまでは、ダミーデータで開発します。

次の1か月(目安)

週	テーマ	ポイント
1週目	実データの棚マスタを作る	図面から座標(X, Y, Z)を起こす作業が実は一番時間がかかります。まず1フロアだけで始めます

週	テーマ	ポイント
2週目	実際の入出庫履歴をつなぐ	CSVのエクスポート、または読み取り専用のDB接続。棚コードの表記ゆれを洗い出します
3週目	見た目の強化	壁・柱・設備を箱で足す。必要になったら Blender で glb を作り、`GLTFLoader` で読み込みます
4週目	運用の準備	配布方法、Runtime の確認、ログ、例外処理、設定ファイル、ユーザーの声の収集

- この先は、在庫数・滞留日数・品質保留などの別指標、リアルタイム更新(Web API + SignalR)、複数拠点への展開が続きます。
- 工場の実データを扱う前に、「この3Dマップを見る人が、何を判断するのか」を現場の人に聞いてください。色の意味(回数か、滞留か、リスクか)が、デザインのいちばん大事な決定になります。

付録: 到達度チェック・用語集・参考リンク

1週間後の到達度チェック

「人に説明できる」を基準にしましょう。説明できない項目は、該当する Day を、1日だけ読み直してください。

- [ ] `position: absolute` と `inset: 0` の意味を説明できる(Day1)
- [ ] `flex` の主軸と交差軸を説明できる(Day1)
- [ ] `import` と `importmap` の役割、`file://` で動かない理由を説明できる(Day2)
- [ ] Scene・Camera・Renderer の関係を説明できる(Day3)
- [ ] 図面座標と Three.js 座標の違い、変換式を説明できる(Day3)
- [ ] `InstancedMesh` が速い理由を説明できる(Day4)
- [ ] `needsUpdate = true` が必要な理由を説明できる(Day4)
- [ ] クリックした棚を判定する流れ(画面座標 → ray → `instanceId` → 棚コード)を説明できる(Day4)
- [ ] WebView2 が別プロセスで、JSON メッセージでしか通信できないことを説明できる(Day5)
- [ ] `ready` ハンドシェイクが必要な理由を説明できる(Day5)
- [ ] データ項目を1つ、CSV から画面まで通せる(Day6)
- [ ] 「色がおかしい」を、4層(CSV・集計・JSON・描画)に切り分けて調べられる(Day6)
- [ ] 自分で選んだ機能を1つ作り、コミットした(Day7)

用語集

用語	意味
DOM	HTMLをプログラムから操作するためのオブジェクトの木
canvas	ピクセルを直接描く絵の領域。Three.jsはここに描く
WebGL	ブラウザでGPUを使って描画する標準API。Three.jsはこれを包んでいる
draw call	CPUからGPUへの「描いて」の命令。回数が多いと重くなる
Geometry / Material / Mesh	形 / 見た目 / その組み合わせ(として置かれる物)
行列(Matrix4)	位置・回転・拡大縮小を1つにまとめた4x4の数字の表
クォータニオン	回転を4つの数で表す方法。数学的に扱いやすい
Raycaster	画面上の点から光線を飛ばして、当たった物を調べる道具
正規化デバイス座標(NDC)	画面を -1〜1 で表す座標。Yが上向き
Box3	軸に平行な直方体(最小点と最大点)
lerp	2つの値を割合で補間する。`a + (b - a) * t`
easing	動きの速さの変化を付けること。最初速く最後ゆっくり等
dispose	GPUメモリを明示的に解放する呼び出し
ES Modules / importmap	`import` でファイルを分ける仕組み / 名前とファイルの対応表
WebView2 / Evergreen Runtime	アプリに埋め込む Edge / PC共通の自動更新されるランタイム
ユーザーデータフォルダ	WebView2 がキャッシュや Cookie を置くフォルダ
仮想ホスト名マッピング	ローカルフォルダを `https://任意の名前/` として見せる設定
ハンドシェイク	「準備できた」と相手が言ってから送る、合図の仕組み

参考(公式ドキュメント)

- Three.js Manual(日本語: Fundamentals): シーン・カメラ・レンダラーの最初の一歩。Day3と同時に読むと理解が深まります
- Three.js Manual(全体の目次): ライト・マテリアル・カメラ・テクスチャなどの解説
- Three.js Docs: `InstancedMesh`、`Raycaster`、`Box3` などの API リファレンス
- Three.js Examples: 動くサンプル集。やりたい表現に近いものを探して、ソースを読みます
- WebView2 の開始(WinForms): Microsoft Learn。初期化の基本
- WebView2 JavaScript リファレンス(`chrome.webview`): JS側のメッセージ API
- MDN: CSS: `position`、`flex` などのプロパティの辞書
- MDN: JavaScript モジュール: `import` / `export` の説明

このガイドのコードの記述は、サンプルプロジェクト(`WarehouseHeatmap`)の内容に基づいています。Three.js はバージョンで API が変わることがあるため、サンプル同梱の r186 系と、ネットの古い記事では名前や引数が異なる場合があります。